Team Profiles & Competency Matrix

- **Ben:** Python, Supabase, Vercel
- **Edwin:** Python, C/C++, Java
- **Nicholas:** C/C++, Java, Python
- **Robin:** Python, C/C++, Next.js, TypeScript, Java
- **Ian:** Python, C++, C, Java

1. Prompt Submitted to the AI

"Our team is building URECAMatch, a web-based matching platform for NTU's URECA program. The system has three roles: Students, Project Supervisors, and Admins.

Core Features:

- *Account and application cycle management.*
- *Preference ranking submission for both students and project supervisors.*
- *An automated matching system that finds the best fit for students and project supervisors based on their ranking submissions.*

Based on the uploaded design materials, recommend a technology stack appropriate for an entry-level software engineering team,considering our collective expertise in Python, Java, C/C++, TypeScript, Next.js, Supabase, and Vercel."

2. Raw AI Tech Stack Recommendation

To build URECAMatch, the AI recommended the following comprehensive industry-standard stack:

- **Frontend:** Next.js (React framework) with TypeScript, styled with Tailwind CSS and Redux Toolkit for global state management.
- **Backend:** Microservices architecture using Node.js/Express.js (for user and cycle management) and Python/FastAPI (specifically for the Gale-Shapley matching engine and NLP semantic recommendation engine).
- **Databases:** PostgreSQL (Primary relational DB for user profiles, listings, rankings) + Redis (for caching listings and handling notification queues).
- **Smart Integrations:**
    - **Auth:** OpenID Connect (OIDC) wrapper integrated with NTU SSO.
    - **AI Recommendations:** SentenceTransformers library hosted on an external AWS SageMaker endpoint.
    - **Scheduler:** Cron job calling the Google Calendar and Outlook Calendar APIs.
- **DevOps & Hosting:** Docker containerization for each microservice, managed via Kubernetes (EKS), deployed on AWS (EC2, RDS, ElastiCache, S3), with a CI/CD pipeline built via GitHub Actions.

3. Team Critique & Software Engineering Evaluation

Our team evaluated this recommendation against our actual competency and limitations to avoid over engineering and maximize development speed.

Our team accepted using an external email service (Resend API) for outbound system alerts. This avoids the heavy setup and spam-rejection risks of managing a local mail server.
It easily satisfies our 15-minute notification speed requirement while allowing us to send automated .ics calendar files for supervisor meeting invites without extra infrastructure costs.